

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности
Направление: 02.03.01 Математика и компьютерные науки

Математическая логика и теория формальных языков
ОТЧЁТ О ВЫПОЛНЕНИИ ЛАБОРАТОРНОЙ
РАБОТЫ №4

«Доработка компилятора языка MiLan»

Вариант 28

Студент,
группы 5130201/30102

_____ Михайлова А. А.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Строки и операции со строками в различных языках программирования	4
1.2 Описание языка MiLan	5
1.3 Устройство компилятора	5
1.4 Грамматика языка SMilan	7
1.5 Грамматика языка Милан в расширенной форме Бэкуса-Наура после доработки компилятора	9
1.6 Синтаксическая диаграмма грамматики	10
2 Особенности реализации	14
2.1 Изменения компилятора	14
2.1.1 Добавление токенов	14
2.1.2 Расширения codegen	16
2.1.3 Изменения parser	18
2.2 Изменения виртуальной машины	21
2.2.1 Модификация vm	21
2.3 Модификация vmlex	27
2.4 Модификация vmparse	27
3 Результаты работы программы	29
3.1 Конкатенация строк	29
3.2 Длина строки	29
3.3 Реверс	30
3.4 Некорректная программа	30
3.5 Некорректный resize с -1	31
Заключение	32
Список использованной литературы	33

Введение

Лабораторная работа заключается в доработке компилятора и виртуальной машины языка MiLan согласно варианту.

Вариант 28:

Операции со строками: конкатенация, поиск подстроки, замена подстроки, реверс, длина, `resize`.

Для выполнения поставленной задачи необходимо:

- Расширить форму Бэкуса-Наура контекстно-свободной грамматикой языка MiLan;
- Построить синтаксическую диаграмму грамматики для расширенной версии языка MiLan;
- Расширить компилятор и виртуальную машину поддержкой строк и операций над ними.

1 Математическое описание

1.1 Строки и операции со строками в различных языках программирования

При разработке модуля работы со строками проводился анализ подходов, используемых в изученных языках программирования: C++, Python, Java и Haskell.

В большинстве современных языков высокого уровня строки реализуются как неизменяемые или изменяемые последовательности символов с использованием специализированных классов и библиотек:

- В C++ строки представляются классом `std::string` из стандартной библиотеки. Класс инкапсулирует динамический массив символов, обеспечивает автоматическое управление памятью и предоставляет методы для конкатенации, поиска, замены и других операций.
- В Python строки являются неизменяемыми объектами класса `str`, хранящими последовательность символов Unicode. Операции конкатенации, среза, поиска и замены возвращают новые объекты строк.
- В Java используются два класса: неизменяемый `String` и изменяемый `StringBuilder/StringBuffer`. Класс `String` хранит массив символов `char[]` и обеспечивает пул строк для оптимизации памяти.
- В Haskell строки представляются как списки символов `[Char]`. Благодаря функциональной природе языка, строки неизменяемы, а операции над ними возвращают новые списки.

Несмотря на различия в синтаксисе, структура данных во всех перечисленных языках едина: строка представляет собой упорядоченный набор символов. Различия заключаются в управлении памятью и неизменяемости: в C++, Python и Java строки инкапсулируются в классы с автоматическим управлением памятью, в Haskell строки реализуются как неизменяемые списки.

Особенность реализации в языке Милан.

Виртуальная машина языка «Милан» написана на языке C и работает по принципу стековой машины (Р-код). В такой среде нет поддержки классов, объектов или сборщика мусора. Вся работа ведётся непосредственно с указателями и примитивными типами данных.

В связи с этим реализация строк в данной работе максимально приближена к низкоуровневому стилю языка C. Строки хранятся в динамической памяти (куче) как массивы символов (`char*`), а в структурах виртуальной машины хранятся только указатели на них:

1. **Стек строк** (`vm_string_stack`) — массив указателей `char*`. При загрузке строки на стек создаётся её копия в куче с помощью `strdup`.
2. **Память строковых переменных** (`vm_string_memory`) — массив указателей `char*`, где индекс соответствует адресу переменной. При присваивании (`STORE_STR`) старый блок памяти освобождается (`free`), а указатель заменяется на новый.
3. **Пул строковых констант** (`vm_string_constants`) — массив указателей на неизменяемые строковые литералы, загружаемые из исходного кода.

1.2 Описание языка MiLan

Язык Милан — учебный язык программирования. Программа на Милане представляет собой последовательность операторов, заключенных между ключевыми словами `begin` и `end`. Операторы отделяются друг от друга точкой с запятой. После последнего оператора в блоке точка с запятой не ставится. Компилятор `CMilan` не учитывает регистр символов в именах переменных и ключевых словах. В базовую версию языка Милан входят следующие конструкции: константы, идентификаторы, арифметические операции над целыми числами, операторы чтения чисел со стандартного ввода и печати чисел на стандартный вывод, оператор присваивания, условный оператор, оператор цикла с предусловием. Программа может содержать комментарии.

1.3 Устройство компилятора

Компилятор `CMilan` преобразует программы на языке Милан в последовательность команд виртуальной машины Милана.

Исполняемый файл компилятора называется `cmilan` (в операционных системах семейства Unix) или `cmilan.exe` (в Windows). Компилятор имеет интерфейс командной строки. Результатом работы компилятора является либо последовательность команд для виртуальной машины Милана, которая печатается на стандартный вывод, либо набор сообщений о синтаксических ошибках, которые выводятся в стандартный поток ошибок.

Чтобы сохранить генерируемую компилятором программу для виртуальной машины в файл, достаточно перенаправить в этот файл стандартный поток вывода.

Компилятор `CMilan` включает три компонента:

1. лексический анализатор;
2. синтаксический анализатор;
3. генератор команд виртуальной машины Милана.

Языки реализации компонентов:

- Лексический и синтаксический анализаторы, генератор кода реализованы на языке C++.
- Виртуальная машина реализована на языке C.
- Генераторы лексического и синтаксического анализаторов виртуальной машины:
 - Файл `vmlex.l` написан на языке спецификации Flex (Lex) — доменно-ориентированном языке для описания лексических правил с использованием регулярных выражений. Встроенные действия записываются на языке C.
 - Файл `vmparse.y` написан на языке спецификации Bison (Yacc) — доменно-ориентированном языке для описания контекстно-свободных грамматик в форме Бэкуса-Наура. Семантические действия также записываются на C.

Лексический анализатор посимвольно читает из входного потока текст программы и преобразует группы символов в лексемы (терминальные символы грамматики языка Милан). При этом он отслеживает номер текущей строки, который используется синтаксическим анализатором при формировании сообщений об ошибках. Также лексический анализатор удаляет пробельные символы и комментарии. При формировании лексем анализатор идентифицирует ключевые слова и последовательности символов (например, оператор присваивания), а также определяет значения числовых констант и имена переменных. Эти значения становятся значениями атрибутов, связанных с лексемами.

Синтаксический анализатор читает сформированную лексическим анализатором последовательность лексем и проверяет ее соответствие грамматике языка Милан. Для этого используется метод рекурсивного спуска. Если в процессе синтаксического анализа обнаруживается ошибка, анализатор формирует сообщение об ошибке, включающее сведения об ошибке и номер строки, в которой она возникла. В процессе анализа программы синтаксический анализатор генерирует набор машинных команд, соответствующие каждой конструкции языка.

Генератор кода представляет собой служебный компонент, ответственный за формирование внешнего представления генерируемого кода. Генератор поддерживает буфер команд и предоставляет синтаксическому анализатору набор функций, позволяющий записать указанную команду по определенному адресу. После того, как синтаксический анализатор заканчивает формирование программы, генератор кода используется для печати на стандартный вывод отсортированной по возрастанию адресов последовательности инструкций.

Все три компонента объединяются «драйвером» — управляющей программой компилятора. Драйвер анализирует аргументы командной строки, инициализирует синтаксический анализатор и вызывает его метод `parse()`, запуская процесс трансляции.

СМіlan является примером простейшего однопроходного компилятора, в котором синтаксический анализ и генерация кода выполняются совместно.

1.4 Грамматика языка СМіlan

Грамматика языка СМіlan является контекстно-свободной, что соответствует 2 типу по иерархии грамматик Хомского.

Порождающая грамматика Хомского определяется как четверка $G = \langle T, N, S, R \rangle$, где:

- T — конечное множество терминалов (слова языка);
- N — конечное множество нетерминалов ($T \cap N = \emptyset$);
- $S \in N$ — начальный нетерминал;
- R — множество правил вывода вида $\alpha \rightarrow \beta$, где α и β — цепочки над словарём $T \cup N$.

В случае контекстно-свободной грамматики:

- Все правила имеют вид:

$$A \rightarrow \alpha$$

где $A \in N$ (одиначный нетерминал), а $\alpha \in (N \cup T)^*$.

- Правые части могут содержать любые комбинации терминалов и нетерминалов, но не зависят от контекста.

В рамках лабораторной работы грамматика языка СМіlan задаётся четвёркой $G = \langle T, N, S, R \rangle$, где:

- $T = \{\text{begin, end, if, then, else, fi, while, do, od, write, read, strcat, strfind, strreplace, strrev, strlen, strresize, writestr, readstr, string, :=, +, -, *, /, =, !=, <, <=, >, >=, (,), ;, ,, a, b, c, d, e, f, g, h, i, j, k, l, m, n, o, p, q, r, s, t, u, v, w, x, y, z, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9}\}$
- $N = \{\text{program, statementList, statement, expression, term, factor, relation, addop, mulop, cmp, ident, number, letter, digit, stringExpression, stringFactor}\}$
- $S = \{\text{program}\}$

• R:

- $\langle \text{program} \rangle \rightarrow \text{'begin' } \langle \text{statementList} \rangle \text{'end'}$
- $\langle \text{statementList} \rangle \rightarrow \langle \text{statement} \rangle \text{' ;' } \langle \text{statementList} \rangle \mid \epsilon$
- $\langle \text{statement} \rangle \rightarrow \langle \text{ident} \rangle \text{' :=' } \langle \text{expression} \rangle$
 - $\mid \langle \text{ident} \rangle \text{' :=' } \langle \text{stringExpression} \rangle$
 - $\mid \text{'if' } \langle \text{relation} \rangle \text{'then' } \langle \text{statementList} \rangle \text{'fi'}$
 - $\mid \text{'if' } \langle \text{relation} \rangle \text{'then' } \langle \text{statementList} \rangle \text{'else' } \langle \text{statementList} \rangle \text{'fi'}$
 - $\mid \text{'while' } \langle \text{relation} \rangle \text{'do' } \langle \text{statementList} \rangle \text{'od'}$
 - $\mid \text{'write' ' (' } \langle \text{expression} \rangle \text{')'}$
 - $\mid \text{'writestr' ' (' } \langle \text{stringExpression} \rangle \text{')'}$
- $\langle \text{expression} \rangle \rightarrow \langle \text{term} \rangle \mid \langle \text{expression} \rangle \langle \text{addop} \rangle \langle \text{term} \rangle$
- $\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \mid \langle \text{term} \rangle \langle \text{mulop} \rangle \langle \text{factor} \rangle$
- $\langle \text{factor} \rangle \rightarrow \langle \text{ident} \rangle \mid \langle \text{number} \rangle \mid \text{' (' } \langle \text{expression} \rangle \text{')' } \mid \text{'read'}$
 - $\mid \text{'strlen' ' (' } \langle \text{stringExpression} \rangle \text{')'}$
 - $\mid \text{'strfind' ' (' } \langle \text{stringExpression} \rangle \text{' , ' } \langle \text{stringExpression} \rangle \text{')'}$
- $\langle \text{stringExpression} \rangle \rightarrow \langle \text{stringFactor} \rangle$
- $\langle \text{stringFactor} \rangle \rightarrow \text{'string'}$
 - $\mid \langle \text{ident} \rangle$
 - $\mid \text{'readstr'}$
 - $\mid \text{'strcat' ' (' } \langle \text{stringExpression} \rangle \text{' , ' } \langle \text{stringExpression} \rangle \text{')'}$
 - $\mid \text{'strrev' ' (' } \langle \text{stringExpression} \rangle \text{')'}$
 - $\mid \text{'strreplace' ' (' } \langle \text{stringExpression} \rangle$
 - $\text{' , ' } \langle \text{stringExpression} \rangle \text{' , ' } \langle \text{stringExpression} \rangle \text{')'}$
 - $\mid \text{'strresize' ' (' } \langle \text{stringExpression} \rangle \text{' , ' } \langle \text{expression} \rangle \text{')'}$
- $\langle \text{relation} \rangle \rightarrow \langle \text{expression} \rangle \langle \text{cmp} \rangle \langle \text{expression} \rangle$
- $\langle \text{addop} \rangle \rightarrow \text{'+' } \mid \text{'-'}$
- $\langle \text{mulop} \rangle \rightarrow \text{'*'} \mid \text{'/'}$
- $\langle \text{cmp} \rangle \rightarrow \text{'=' } \mid \text{'!=' } \mid \text{'<' } \mid \text{'<=' } \mid \text{'>' } \mid \text{'>='}$
- $\langle \text{ident} \rangle \rightarrow \langle \text{letter} \rangle \mid \langle \text{ident} \rangle \langle \text{letter} \rangle \mid \langle \text{ident} \rangle \langle \text{digit} \rangle$
- $\langle \text{number} \rangle \rightarrow \langle \text{digit} \rangle \mid \langle \text{number} \rangle \langle \text{digit} \rangle$
- $\langle \text{letter} \rangle \rightarrow \text{'a' } \mid \dots \mid \text{'z' } \mid \text{'A' } \mid \dots \mid \text{'Z'}$
- $\langle \text{digit} \rangle \rightarrow \text{'0' } \mid \text{'1' } \mid \dots \mid \text{'9'}$

Грамматика языка Милан является LL(1)-грамматикой, что позволяет использовать метод рекурсивного спуска с одним символом.

Грамматикой рекурсивного спуска называется LL(1)-грамматика, которая представляется такими синтаксическими диаграммами для каждого нетерминала, что в любом разветвлении каждой синтаксической диаграммы выбор альтернативного пути может быть сделан на основе анализа одного очередного терминального символа.

1.5 Грамматика языка Милан в расширенной форме Бэкуса-Наура после доработки компилятора

Форма Бэкуса-Наура (БНФ) — форма представления контекстно-свободных формальных грамматик, в которой правые части продукций с одинаковой левой частью объединены.

Расширенная БНФ включает «регулярные» конструкции: итерацию a^* , необязательность $[a]$, группировку $(a | b)$.

Изменения, необходимые для доработки компилятора, выделены синим цветом.

$$\begin{aligned}
 \langle program \rangle &::= 'begin' \langle statementList \rangle 'end' \\
 \langle statementList \rangle &::= \langle statement \rangle ';' \langle statementList \rangle \mid \epsilon \\
 \langle statement \rangle &::= \langle ident \rangle := \langle expression \rangle \\
 &\quad | \langle ident \rangle := \langle stringExpression \rangle \\
 &\quad | 'if' \langle relation \rangle 'then' \langle statementList \rangle ['else' \langle statementList \rangle] \\
 &\quad 'fi' \\
 &\quad | 'while' \langle relation \rangle 'do' \langle statementList \rangle 'od' \\
 &\quad | 'write' '(' \langle expression \rangle ')', \\
 &\quad | 'writestr' '(' \langle stringExpression \rangle ')', \\
 \langle expression \rangle &::= \langle term \rangle \{ \langle addop \rangle \langle term \rangle \} \\
 \langle term \rangle &::= \langle factor \rangle \{ \langle mulop \rangle \langle factor \rangle \} \\
 \langle factor \rangle &::= \langle ident \rangle \mid \langle number \rangle \mid \langle factor \rangle \mid (\langle expression \rangle) \mid 'read' \\
 &\quad | 'strlen' '(' \langle stringExpression \rangle ')', \\
 &\quad | 'strfind' '(' \langle stringExpression \rangle ',' \langle stringExpression \rangle ')', \\
 \langle stringExpression \rangle &::= \langle stringFactor \rangle \\
 \langle stringFactor \rangle &::= 'string' \\
 &\quad | \langle ident \rangle \\
 &\quad | 'readstr' \\
 &\quad | 'strcat' '(' \langle stringExpression \rangle ',' \langle stringExpression \rangle ')',
 \end{aligned}$$

$\mid$ 'strrev' '(' $\langle stringExpression \rangle$ ')'
 $\mid$ 'strreplace' '(' $\langle stringExpression \rangle$ ',' $\langle stringExpression \rangle$ ',' $\langle stringExpression \rangle$ ')'
 $\mid$ 'strresize' '(' $\langle stringExpression \rangle$ ',' $\langle expression \rangle$ ')'
 $\langle relation \rangle ::= \langle expression \rangle \langle cmp \rangle \langle expression \rangle$
 $\langle addop \rangle ::= '+' \mid '-'$
 $\langle mulop \rangle ::= '*' \mid '/'$
 $\langle cmp \rangle ::= '=' \mid '!=' \mid '<' \mid '<=' \mid '>' \mid '>='$
 $\langle ident \rangle ::= \langle letter \rangle \{ \langle letter \rangle \mid \langle digit \rangle \}$
 $\langle number \rangle ::= \langle digit \rangle \{ \langle digit \rangle \}$
 $\langle letter \rangle ::= 'a' \mid 'b' \mid \dots \mid 'z' \mid 'A' \mid 'B' \mid \dots \mid 'Z'$
 $\langle digit \rangle ::= '0' \mid '1' \mid \dots \mid '9'$

1.6 Синтаксическая диаграмма грамматики

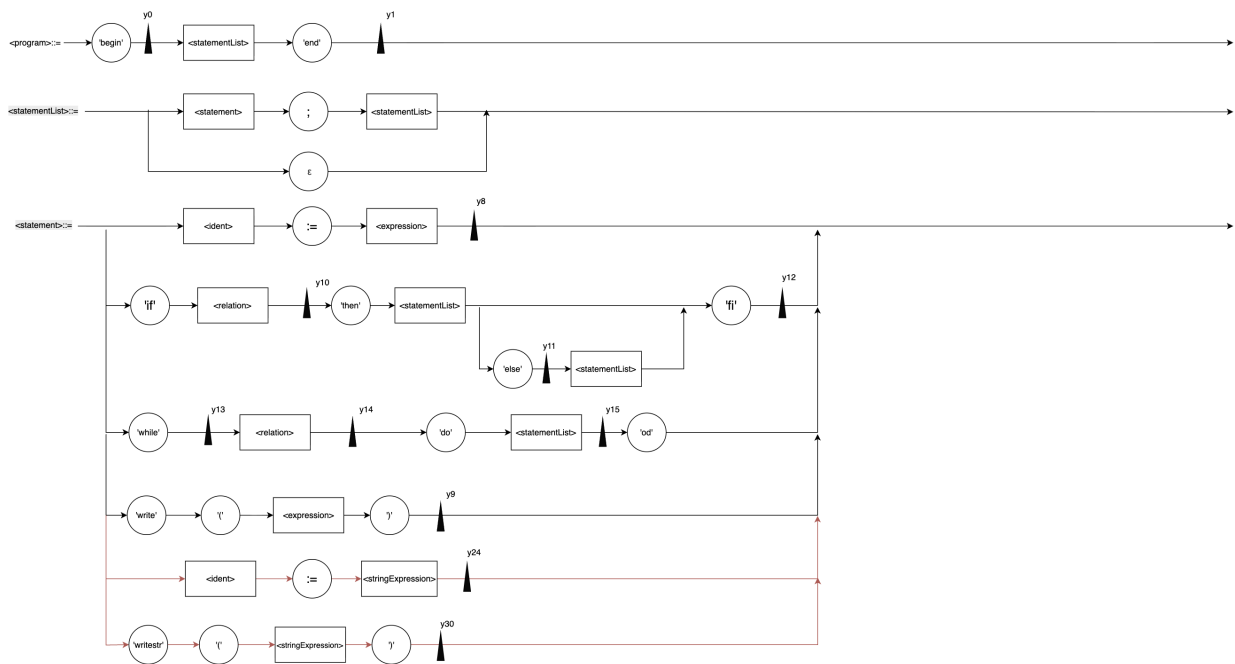


Рис. 1: Синтаксическая диаграмма

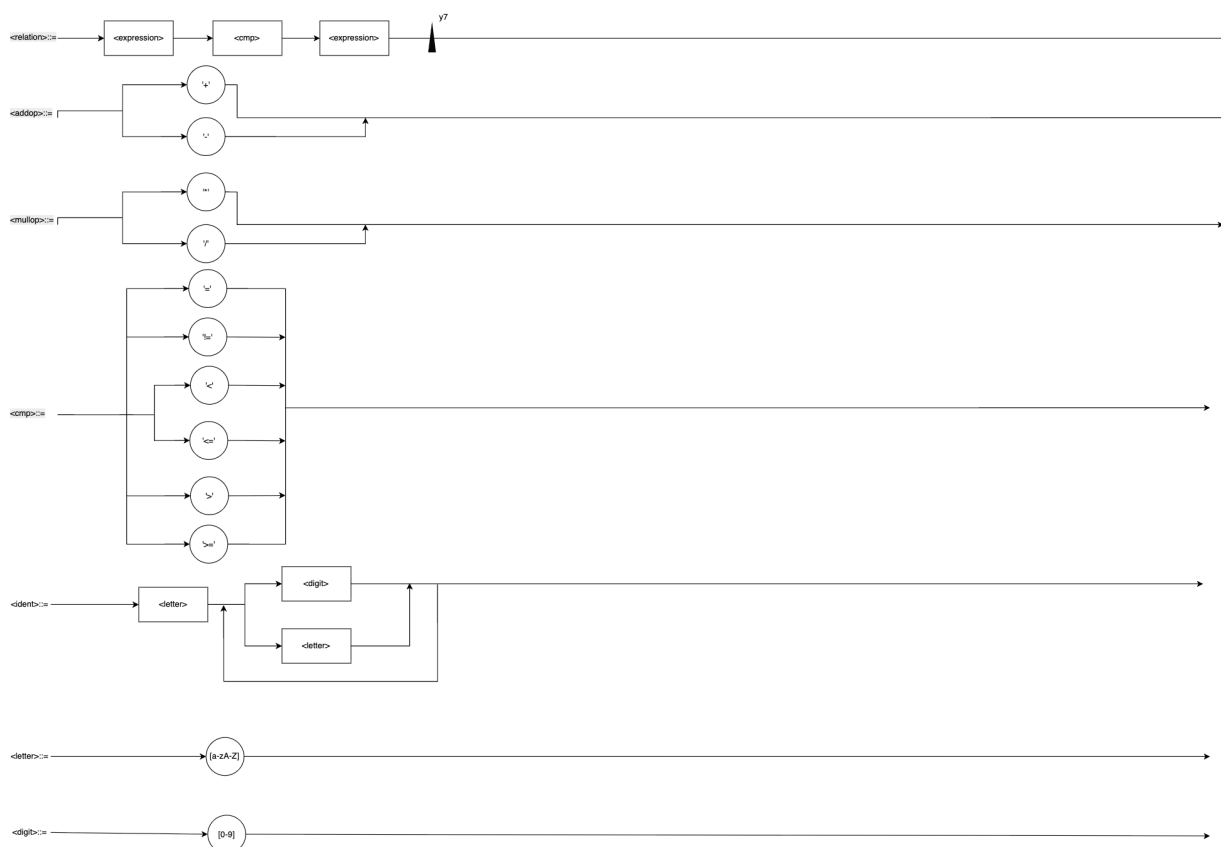


Рис. 4: Синтаксическая диаграмма

Ниже представлены семантические действия, необходимые для генерации кода.

- y_0 Инициализация счётчика команд: $C := 0$;
- y_1 Генерация команды остановки: STOP
- y_2 Загрузка переменной: LOAD addr
- y_3 Загрузка константы: PUSH value
- y_5 Операция умножения/деления: MULT или DIV
- y_6 Операция сложения/вычитания: ADD или SUB
- y_7 Сравнение значений: COMPARE
- y_8 Присваивание: STORE addr
- y_9 Вывод на экран: PRINT
- y_{10} Резервирование условного перехода: JUMP_NO
- y_{11} Резервирование перехода через ветку else: JUMP

- y_{12} Заполнение адресов переходов для `if`: заполнение `JUMP_NO` и `JUMP`
- y_{13} Запоминание адреса начала проверки: `loopStartAddr := C`
- y_{14} Резервирование выхода из цикла: `JUMP_NO`
- y_{15} Возврат к условию и завершение цикла: `JUMP loopStartAddr` и заполнение `JUMP_NO`
- y_{16} Начало обработки цикла `for`: вход в ветку разбора
- y_{17} Сохранение начального значения и адреса проверки: `STORE initVarAddr, loopStartAddr := C`
- y_{18} Резервирование ячеек для переходов: `JUMP_NO exit, JUMP body`
- y_{19} Генерация обновления и возврата к условию: `STORE updateVarAddr, JUMP loopStartAddr`
- y_{20} Замыкание цикла и заполнение выходов: `JUMP updateAddr`, заполнение зарезервированных ячеек
- y_{21} Снять строку с строкового стека, положить длину на целочисленный: `STRLEN`
- y_{22} Снять подстроку и строку с строкового стека, положить позицию на целочисленный: `STRFIND`
- y_{23} Загрузка строки: `PUSH_STR idx`
- y_{24} Загрузка строки на строковый стек: `LOAD_STR addr`
- y_{25} Прочитать строку и положить на стек: `INPUT_STR`
- y_{26} Удалить 2 строки со стека, положить конкатенацию: `STRCAT`
- y_{27} Заменить на перевернутую: `STRREV`
- y_{28} Удалить `new`, `old`, `src`, положить результат: `STRREPLACE`
- y_{29} Снять `n` с целочисленного, строку со строкового, положить на строковой: `STRRESIZE`
- y_{30} Вывод на экран: `PRINT_STR`

2 Особенности реализации

В ходе выполнения лабораторной работы компилятор языка Милан и виртуальная машина были расширены для поддержки нового типа данных — строка и операций над ним.

2.1 Изменения компилятора

2.1.1 Добавление токенов

Для инициализации строк и добавления операций над строками в enum Token были добавлены следующие токены:

```
1 enum Token {
2     T_EOF,
3     T_ILLEGAL,
4     T_IDENTIFIER,
5     T_NUMBER,
6     T_BEGIN,
7     T_END,
8     T_IF,
9     T_THEN,
10    T_ELSE,
11    T_FI,
12    T_WHILE,
13    T_DO,
14    T_OD,
15    T_WRITE,
16    T_READ,
17    T_ASSIGN,
18    T_ADDOP,
19    T_MULOP,
20    T_CMP,
21    T_LPAREN,
22    T_RPAREN,
23    T_SEMICOLON,
24    T_COMMA,
25    T_STRING,
26    T_STRCAT,
27    T_STRFIND,
28    T_STRREPLACE,
29    T_STRREV,
30    T_STRLEN,
31    T_STRRESIZE,
32    T_WRITESTR,
33    T_READSTR,
34 };
```

В массив tokenNames_ добавлены строковые представления новых токенов.

```
1 static const char * tokenNames_[] = {
2     "end of file",
3     "illegal token",
4     "identifier",
5     "number",
6     "'BEGIN'",
```

```

7  " 'END' ",
8  " 'IF' ",
9  " 'THEN' ",
10 " 'ELSE' ",
11 " 'FI' ",
12 " 'WHILE' ",
13 " 'DO' ",
14 " 'OD' ",
15 " 'WRITE' ",
16 " 'READ' ",
17 " ':' ",
18 " '+' or '-' ",
19 " '*' or '/' ",
20 "comparison operator",
21 " '(',
22 " ')'",
23 " ';' ",
24 " ',' ",
25 "string literal",
26 " 'strcat'",
27 " 'strfind'",
28 " 'strreplace'",
29 " 'strrev'",
30 " 'strlen'",
31 " 'strresize'",
32 " 'writestr'",
33 " 'readstr'",
34 };

```

В конструкторе класса Scanner добавлены новые ключевые слова.

```

1  explicit Scanner(const string& fileName, istream& input)
2  : fileName_(fileName), lineNumber_(1), input_(input)
3  {
4      keywords_["begin"] = T_BEGIN;
5      keywords_["end"] = T_END;
6      keywords_["if"] = T_IF;
7      keywords_["then"] = T_THEN;
8      keywords_["else"] = T_ELSE;
9      keywords_["fi"] = T_FI;
10     keywords_["while"] = T_WHILE;
11     keywords_["do"] = T_DO;
12     keywords_["od"] = T_OD;
13     keywords_["write"] = T_WRITE;
14     keywords_["read"] = T_READ;
15     keywords_["strcat"] = T_STRCAT;
16     keywords_["strfind"] = T_STRFIND;
17     keywords_["strreplace"] = T_STRREPLACE;
18     keywords_["strrev"] = T_STRREV;
19     keywords_["strlen"] = T_STRLEN;
20     keywords_["strresize"] = T_STRRESIZE;
21     keywords_["writestr"] = T_WRITESTR;
22     keywords_["readstr"] = T_READSTR;
23
24     nextChar();
25 }

```

В nextToken() добавлены обработчики для символов «"» и «,».

```

1  case '"': {
2      nextChar();

```

```

3   string buffer;
4   while(ch_ != '"' && !input_.eof()) {
5       if(ch_ == '\\') {
6           nextChar();
7           switch(ch_) {
8               case 'n':  buffer += '\n'; break;
9               case 't':  buffer += '\t'; break;
10              case '"':  buffer += '"';  break;
11              case '\\': buffer += '\\'; break;
12              default:   buffer += ch_;  break;
13          }
14      } else {
15          buffer += ch_;
16      }
17      nextChar();
18  }
19  if(!input_.eof()) nextChar();
20  token_ = T_STRING;
21  stringValue_ = buffer;
22  break;
23 }
24
25 case ',':
26 token_ = T_COMMA;
27 nextChar();
28 break;

```

2.1.2 Расширения codegen

В enum Instruction добавлены новые инструкции.

```

1  enum Instruction
2  {
3      NOP,
4      STOP,
5      LOAD,
6      STORE,
7      BLOAD,
8      BSTORE,
9      PUSH,
10     POP,
11     DUP,
12     ADD,
13     SUB,
14     MULT,
15     DIV,
16     INVERT,
17     COMPARE,
18     JUMP_YES,
19     JUMP_NO,
20     INPUT,
21     PRINT,
22     PUSH_STR,
23     LOAD_STR,
24     STORE_STR,
25     STRCAT,
26     STRFIND,
27     STRREPLACE,
28     STRREV,

```



```

29     STRLEN,
30     STRRESIZE,
31     PRINT_STR,
32     INPUT_STR
33 };

```

В методе `Command::print` добавлены ветви `case` для новых инструкций, обеспечивающие их текстовое представление при печати листинга программы.

```

1  case PUSH_STR:
2  os << "PUSH_STR\t" << arg_;
3  break;
4  case LOAD_STR:
5  os << "LOAD_STR\t" << arg_;
6  break;
7  case STORE_STR:
8  os << "STORE_STR\t" << arg_;
9  break;
10 case STRCAT:
11 os << "STRCAT";
12 break;
13 case STRFIND:
14 os << "STRFIND";
15 break;
16 case STRREPLACE:
17 os << "STRREPLACE";
18 break;
19 case STRREV:
20 os << "STRREV";
21 break;
22 case STRLEN:
23 os << "STRLEN";
24 break;
25 case STRRESIZE:
26 os << "STRRESIZE";
27 break;
28 case PRINT_STR:
29 os << "PRINT_STR";
30 break;
31 case INPUT_STR:
32 os << "INPUT_STR";
33 break;

```

Добавлен новый метод `addStringConstant` для сохранения всех строк в коде в массив `stringConstants_`. С помощью него мы можем ссылаться на индексы при работе со строками, заменяет строковые данные.

Вход: новая строка

Выход: индекс добавленной строки

```

1  int CodeGen::addStringConstant(const string& s)
2  {
3      int index = (int)stringConstants_.size();
4      stringConstants_.push_back(s);
5      return index;
6  }

```

Изменён метод `flush`: добавлен вывод пула строковых констант.

```

1 void CodeGen::flush()
2 {
3     if(!stringConstants_.empty()) {
4         output_ << ".strings " << stringConstants_.size() << endl;
5         for(int i = 0; i < (int)stringConstants_.size(); ++i) {
6             output_ << i << ":\t" << stringConstants_[i] << endl;
7         }
8         output_ << ".code" << endl;
9     }
10
11     int count = (int)commandBuffer_.size();
12     for(int address = 0; address < count; ++address) {
13         commandBuffer_[address].print(address, output_);
14     }
15     output_.flush();
16 }

```

2.1.3 Изменения parser

В класс были добавлены таблица строковых переменных и целочисленная переменная-адрес следующей новой строковой переменной.

```

1 typedef map<string, int> StrVarTable;
2 StrVarTable strVariables_;
3 int lastStrVar_;

```

В функции `statement()` изменена ветка `T_IDENTIFIER` для правильной записи переменной или в таблицу строк, или в таблицу чисел и вызова нужного метода для обработки и добавлена ветка `T_WRITESTR` для печати строк в код для виртуальной машины.

```

1 if(see(T_IDENTIFIER)) {
2     string varName = scanner_>getStringValue();
3     next();
4     mustBe(T_ASSIGN);
5
6     bool rhsIsString =
7     see(T_STRING)      || see(T_STRCAT)    || see(T_STRREV) ||
8     see(T_STRREPLACE) || see(T_STRRESIZE)|| see(T_READSTR)||
9     (see(T_IDENTIFIER) && isStringVariable(scanner_>getStringValue()));
10
11     if(rhsIsString) {
12         int varAddr = findOrAddStringVariable(varName);
13         stringExpression();
14         codegen_>emit(STORE_STR, varAddr);
15     } else {
16         int varAddr = findOrAddVariable(varName);
17         expression();
18         codegen_>emit(STORE, varAddr);
19     }
20 }

```

```

1 else if(match(T_WRITESTR)) {
2     mustBe(T_LPAREN);
3     stringExpression();
4     mustBe(T_RPAREN);

```

```

5 |   codegen_ ->emit(PRINT_STR);
6 | }

```

В функции factor() добавлены strlen для вычисления длины строки и strstr для поиска подстроки(если подстрока не найдена возвращаем -1).

```

1 | else if(match(T_STRLEN)) {
2 |     mustBe(T_LPAREN);
3 |     stringExpression();
4 |     mustBe(T_RPAREN);
5 |     codegen_ ->emit(STRLEN);
6 | }
7 | else if(match(T_STRFIND)) {
8 |     mustBe(T_LPAREN);
9 |     stringExpression();
10 |    mustBe(T_COMMA);
11 |    stringExpression();
12 |    mustBe(T_RPAREN);
13 |    codegen_ ->emit(STRFIND);
14 | }

```

Добавлены реализации новых методов

- stringExpression() – точка входа для разбора любого строкового выражения.

Вход: строка

Выход: соответствующая операции инструкция

```

1 | void Parser::stringExpression()
2 | {
3 |     stringFactor();
4 | }

```

- stringFactor() – реализует операции конкатенации, замены подстроки, реверса, resize.

Вход: строка

Выход: соответствующая операции инструкция

```

1 | void Parser::stringFactor()
2 | {
3 |     if(see(T_STRING)) {
4 |         int idx = codegen_ ->addStringConstant(scanner_ ->getStringValue());
5 |         next();
6 |         codegen_ ->emit(PUSH_STR, idx);
7 |     }
8 |     else if(see(T_IDENTIFIER) && isStringVariable(scanner_ ->getStringValue())) {
9 |         int addr = findOrAddStringVariable(scanner_ ->getStringValue());
10 |        next();
11 |        codegen_ ->emit(LOAD_STR, addr);
12 |    }
13 |    else if(match(T_READSTR)) {
14 |        codegen_ ->emit(INPUT_STR);
15 |    }
16 |    else if(match(T_STRCAT)) {

```

```

17     mustBe(T_LPAREN);
18     stringExpression();
19     mustBe(T_COMMA);
20     stringExpression();
21     mustBe(T_RPAREN);
22     codegen_ ->emit(STRCAT);
23 }
24 else if(match(T_STRREV)) {
25     mustBe(T_LPAREN);
26     stringExpression();
27     mustBe(T_RPAREN);
28     codegen_ ->emit(STRREV);
29 }
30 else if(match(T_STRREPLACE)) {
31     mustBe(T_LPAREN);
32     stringExpression();
33     mustBe(T_COMMA);
34     stringExpression();
35     mustBe(T_COMMA);
36     stringExpression();
37     mustBe(T_RPAREN);
38     codegen_ ->emit(STRREPLACE);
39 }
40 else if(match(T_STRRESIZE)) {
41     mustBe(T_LPAREN);
42     stringExpression();
43     mustBe(T_COMMA);
44     expression();
45     mustBe(T_RPAREN);
46     codegen_ ->emit(STRRESIZE);
47 }
48 else {
49     reportError("string expression expected.");
50 }
51 }

```

- `findOrAddStringVariable()` – управляет таблицей строковых переменных, назначает каждой переменной адрес в строковой памяти виртуальной машины.

Вход: строка

Выход: адрес строки

```

1 int Parser::findOrAddStringVariable(const string& var)
2 {
3     StrVarTable::iterator it = strVariables_.find(var);
4     if(it == strVariables_.end()) {
5         strVariables_[var] = lastStrVar_;
6         return lastStrVar_++;
7     } else {
8         return it->second;
9     }
10 }

```

- `isStringVariable()` – проверяет, является ли переменная строкой, чтобы отличать строковые переменные от целочисленных.

Вход: переменная

Выход: true если строка, false если нет

```
1 bool Parser::isStringVariable(const string& name)
2 {
3     return strVariables_.find(name) != strVariables_.end();
4 }
```

2.2 Изменения виртуальной машины

2.2.1 Модификация vm

В enum operation добавлены коды новых команд.

```
1 typedef enum {
2     NOP = 0,
3     STOP,
4     LOAD,
5     STORE,
6     BLOAD,
7     BSTORE,
8     PUSH,
9     POP,
10    DUP,
11    INVERT,
12    ADD,
13    SUB,
14    MULT,
15    DIV,
16    COMPARE,
17    JUMP,
18    JUMP_YES,
19    JUMP_NO,
20    INPUT,
21    PRINT,
22    PUSH_STR,
23    LOAD_STR,
24    STORE_STR,
25    STRCAT,
26    STRFIND,
27    STRREPLACE,
28    STRREV,
29    STRLEN,
30    STRRESIZE,
31    PRINT_STR,
32    INPUT_STR
33 } operation;
```

В opcodes_table расширена таблица добавлением новых кодов.

```
1 opcode_info opcodes_table[] = {
2     {"NOP", 0},
3     {"STOP", 0},
4     {"LOAD", 1},
5     {"STORE", 1},
6     {"BLOAD", 1},
7     {"BSTORE", 1},
8     {"PUSH", 1},
```

```

9   {"POP",      0},
10  {"DUP",      0},
11  {"INVERT",   0},
12  {"ADD",      0},
13  {"SUB",      0},
14  {"MULT",     0},
15  {"DIV",      0},
16  {"COMPARE",  1},
17  {"JUMP",     1},
18  {"JUMP_YES", 1},
19  {"JUMP_NO",  1},
20  {"INPUT",    0},
21  {"PRINT",    0},
22  {"PUSH_STR", 1},
23  {"LOAD_STR", 1},
24  {"STORE_STR", 1},
25  {"STRCAT",    0},
26  {"STRFIND",   0},
27  {"STRREPLACE", 0},
28  {"STRREV",    0},
29  {"STRLEN",    0},
30  {"STRRESIZE", 0},
31  {"PRINT_STR", 0},
32  {"INPUT_STR", 0}
33 };

```

Реализованы новые методы:

- `vm_push_str` – кладёт строку на верх стека для строк, делая собственную копию строки в куче

Вход: строка

Выход: стек с добавленной строкой

```

1 void vm_push_str(const char* s)
2 {
3     if(vm_string_pointer < MAX_STRING_STACK_SIZE) {
4         vm_string_stack[vm_string_pointer++] = strdup(s);
5     } else {
6         vm_error(String_stack_overflow);
7     }
8 }

```

- `vm_pop_str` – забирает строку с вершины стека для строк

Вход: стек строк

Выход: указатель на строку с вершины стека

```

1 char* vm_pop_str()
2 {
3     if(vm_string_pointer > 0) {
4         return vm_string_stack[--vm_string_pointer];
5     } else {
6         vm_error(String_stack_empty);
7         return NULL;
8     }
9 }

```

- `add_string_constant` – заполняет массив строковых констант при загрузке программы.

Вход: индекс, строка

Выход: массив строковых констант

```

1 void add_string_constant(int index, const char* s)
2 {
3     if(index >= 0 && index < MAX_STRING_CONSTANTS) {
4         vm_string_constants[index] = strdup(s);
5         if(index >= vm_string_constants_count)
6             vm_string_constants_count = index + 1;
7     }
8 }

```

- `vm_string_ensure` – гарантирует что ячейка строковой памяти по адресу `addr` существует и не равна `NULL` перед тем как её читать

Вход: адресс ячейки

Выход: `NULL` или `true`

```

1 static void vm_string_ensure(unsigned int addr)
2 {
3     if(addr >= MAX_STRING_MEMORY_SIZE) {
4         vm_error(BAD_DATA_ADDRESS);
5     }
6     if(vm_string_memory[addr] == NULL) {
7         vm_string_memory[addr] = strdup("");
8     }
9 }

```

Добавлены новые case в `vm_run_command`:

- `PUSH_STR` загружает строковый литерал из пула констант на строковый стек;
- `LOAD_STR` загружает значение строковой переменной из памяти на стек;
- `STORE_STR` снимает строку с вершины стека и сохраняет в памяти строковых переменных;
- `STRCAT` склеивает две строки со стека и кладёт результат обратно;
- `STRFIND` ищет подстроку в строке и кладёт целое число (позицию) на целочисленный стек;
- `STRREPLACE` заменяет все вхождения подстроки в строке на новую строку;
- `STRREV` переворачивает строку на вершине стека;

- STRLEN снимает строку со строкового стека и кладёт её длину на целочисленный стек;
- STRRESIZE изменяет длину строки – обрезает или дополняет пробелами;
- PRINT_STR печатает строку с вершины стека на стандартный вывод;
- INPUT_STR читает строку со стандартного ввода и кладёт на строковый стек.

```

1 case PUSH_STR: {
2     if(arg < (unsigned int)vm_string_constants_count
3     && vm_string_constants[arg] != NULL) {
4         vm_push_str(vm_string_constants[arg]);
5     } else {
6         vm_error(BAD_DATA_ADDRESS);
7     }
8     break;
9 }
10
11 case LOAD_STR: {
12     vm_string_ensure(arg);
13     vm_push_str(vm_string_memory[arg]);
14     break;
15 }
16
17 case STORE_STR: {
18     char* s = vm_pop_str();
19     if(s) {
20         if(vm_string_memory[arg]) free(vm_string_memory[arg]);
21         vm_string_memory[arg] = s;
22     }
23     break;
24 }
25
26 case STRCAT: {
27     char* s2 = vm_pop_str();
28     char* s1 = vm_pop_str();
29     if(s1 && s2) {
30         char* result = (char*)malloc(strlen(s1) + strlen(s2) + 1);
31         strcpy(result, s1);
32         strcat(result, s2);
33         vm_push_str(result);
34         free(result);
35         free(s1);
36         free(s2);
37     }
38     break;
39 }
40
41 case STRFIND: {
42     char* sub = vm_pop_str();
43     char* src = vm_pop_str();
44     if(src && sub) {
45         char* found = strstr(src, sub);
46         int pos = (found == NULL) ? -1 : (int)(found - src);
47         vm_push(pos);
48         free(src);

```



```

49     free(sub);
50 }
51 break;
52 }
53
54 case STRREPLACE: {
55     char* new_sub = vm_pop_str();
56     char* old_sub = vm_pop_str();
57     char* src      = vm_pop_str();
58     if(src && old_sub && new_sub) {
59         size_t src_len      = strlen(src);
60         size_t old_len      = strlen(old_sub);
61         size_t new_len      = strlen(new_sub);
62         size_t count = 0;
63         const char* p = src;
64         if(old_len > 0) {
65             while((p = strstr(p, old_sub)) != NULL) {
66                 ++count;
67                 p += old_len;
68             }
69         }
70         size_t result_len = src_len + count * (new_len - old_len) + 1;
71         char* result = (char*)malloc(result_len);
72         char*dst = result;
73         p = src;
74         if(old_len > 0) {
75             const char* found;
76             while((found = strstr(p, old_sub)) != NULL) {
77                 size_t skip = found - p;
78                 memcpy(dst, p, skip);
79                 dst += skip;
80                 memcpy(dst, new_sub, new_len);
81                 dst += new_len;
82                 p = found + old_len;
83             }
84         }
85         strcpy(dst, p);
86         vm_push_str(result);
87         free(result);
88         free(src);
89         free(old_sub);
90         free(new_sub);
91     }
92     break;
93 }
94
95 case STRREV: {
96     char* s = vm_pop_str();
97     if(s) {
98         size_t len = strlen(s);
99         char* r = (char*)malloc(len + 1);
100         for(size_t i = 0; i < len; ++i)
101             r[i] = s[len - 1 - i];
102         r[len] = '\0';
103         vm_push_str(r);
104         free(r);
105         free(s);
106     }
107     break;
108 }

```

```

109
110 case STRLEN: {
111     char* s = vm_pop_str();
112     if(s) {
113         int len = (int)strlen(s);
114         free(s);
115         vm_push(len);
116     }
117     break;
118 }
119
120 case STRRESIZE: {
121     int n = vm_pop();
122     char* s = vm_pop_str();
123     if(s) {
124         if(n < 0) n = 0;
125         char* r = (char*)malloc(n + 1);
126         size_t old_len = strlen(s);
127         if((size_t)n <= old_len) {
128             memcpy(r, s, n);
129         } else {
130             memcpy(r, s, old_len);
131             memset(r + old_len, ' ', n - old_len);
132         }
133         r[n] = '\0';
134         vm_push_str(r);
135         free(r);
136         free(s);
137     }
138     break;
139 }
140
141 case PRINT_STR: {
142     char* s = vm_pop_str();
143     if(s) {
144         fprintf(stdout, "%s\n", s);
145         free(s);
146     }
147     break;
148 }
149
150 case INPUT_STR: {
151     char buf[4096];
152     fprintf(stderr, "> "); fflush(stderr);
153     if(fgets(buf, sizeof(buf), stdin)) {
154         /* убирем завершающий '\n' */
155         size_t len = strlen(buf);
156         if(len > 0 && buf[len-1] == '\n') buf[len-1] = '\0';
157         vm_push_str(buf);
158     } else {
159         vm_error(BAD_INPUT);
160     }
161     break;
162 }

```

Инициализирована строковая память в `vm_init()`.

```

1 void vm_init()
2 {
3     vm_stack_pointer = 0;
4     vm_command_pointer = 0;

```

```

5
6   vm_string_pointer = 0;
7   memset(vm_string_memory, 0, sizeof(vm_string_memory));
8   memset(vm_string_constants, 0, sizeof(vm_string_constants));
9   vm_string_constants_count = 0;
10 }

```

2.3 Модификация vmlex

В vmlex добавлены правила PUSH_STR, LOAD_STR, STORE_STR, STRCAT, STRFIND, STRREPLACE, STRREV, STRRESIZE, PRINT_STR, INPUT_STR для распознавания новых инструкций. Каждое правило возвращает соответствующий токен, который затем обрабатывается парсером vmparse.

```

1  \.strings[ \t]+[0-9]+ {
2      strings_expected = atoi(yytext + 9);
3      strings_read = 0;
4      BEGIN(STRINGS_SECTION);
5  }
6
7  \.code    { BEGIN(INITIAL); }
8
9  <STRINGS_SECTION>[^\n]*\n {
10     int idx = atoi(yytext);
11     char* tab = strchr(yytext, '\t');
12     if(tab) {
13         char val[4096];
14         strncpy(val, tab + 1, sizeof(val) - 1);
15         val[sizeof(val)-1] = '\0';
16         size_t len = strlen(val);
17         if(len > 0 && val[len-1] == '\n') val[len-1] = '\0';
18         add_string_constant(idx, val);
19     }
20     ++strings_read;
21 }
22
23 <STRINGS_SECTION>[ \t\r\n]+
24
25 PUSH_STR    { return T_PUSH_STR;    }
26 LOAD_STR    { return T_LOAD_STR;    }
27 STORE_STR   { return T_STORE_STR;   }
28 STRCAT      { return T_STRCAT;      }
29 STRFIND     { return T_STRFIND;     }
30 STRREPLACE  { return T_STRREPLACE;  }
31 STRREV      { return T_STRREV;      }
32 STRLEN      { return T_STRLEN;      }
33 STRRESIZE   { return T_STRRESIZE;   }
34 PRINT_STR   { return T_PRINT_STR;   }
35 INPUT_STR   { return T_INPUT_STR;   }

```

2.4 Модификация vmparse

Были добавлены объявления токенов.

```

1 %token T_PUSH_STR
2 %token T_LOAD_STR
3 %token T_STORE_STR
4 %token T_STRCAT
5 %token T_STRFIND
6 %token T_STRREPLACE
7 %token T_STRREV
8 %token T_STRLEN
9 %token T_STRRESIZE
10 %token T_PRINT_STR
11 %token T_INPUT_STR

```

В секцию правил грамматики добавлены новые альтернативы для правила `line`, которые позволяют разбирать строки. Каждое правило вызывает функцию `put_command()` для записи соответствующей команды в память виртуальной машины:

```

1 | T_INT T_COLON T_PUSH_STR T_INT { put_command($1, PUSH_STR, $4); }
2 | T_INT T_COLON T_LOAD_STR T_INT { put_command($1, LOAD_STR, $4); }
3 | T_INT T_COLON T_STORE_STR T_INT { put_command($1, STORE_STR, $4); }
4 | T_INT T_COLON T_STRCAT { put_command($1, STRCAT, 0); }
5 | T_INT T_COLON T_STRFIND { put_command($1, STRFIND, 0); }
6 | T_INT T_COLON T_STRREPLACE { put_command($1, STRREPLACE, 0); }
7 | T_INT T_COLON T_STRREV { put_command($1, STRREV, 0); }
8 | T_INT T_COLON T_STRLEN { put_command($1, STRLEN, 0); }
9 | T_INT T_COLON T_STRRESIZE { put_command($1, STRRESIZE, 0); }
10 | T_INT T_COLON T_PRINT_STR { put_command($1, PRINT_STR, 0); }
11 | T_INT T_COLON T_INPUT_STR { put_command($1, INPUT_STR, 0); }

```

3 Результаты работы программы

Для проверки корректности работы были реализованы несколько тестовых программ, направленных на проверку различных сценариев.

3.1 Конкатенация строк

Текст исходного кода:

```
1 begin
2 s := "Hello";
3 t := ", world!";
4 result := strcat(s, t);
5 writestr(result)
6 end
```

Текст out-файла:

```
1 .strings 2
2 0: Hello
3 1: , world!
4 .code
5 0: PUSH_STR 0
6 1: STORE_STR 0
7 2: PUSH_STR 1
8 3: STORE_STR 1
9 4: LOAD_STR 0
10 5: LOAD_STR 1
11 6: STRCAT
12 7: STORE_STR 2
13 8: LOAD_STR 2
14 9: PRINT_STR
15 10: STOP
```

Вывод программы:

Hello, world!

3.2 Длина строки

Текст исходного кода:

```
1 begin
2 s := "Hello";
3 n := strlen(s);
4 write(n)
5 end
```

Текст out-файла:

```
1 .strings 1
2 0: Hello
3 .code
4 0: PUSH_STR 0
5 1: STORE_STR 0
6 2: LOAD_STR 0
7 3: STRLEN
```

```

8 4:  STORE 0
9 5:  LOAD  0
10 6:  PRINT
11 7:  STOP

```

Вывод программы:

5

3.3 Реверс

Текст исходного кода:

```

1 begin
2 s := "Hello";
3 result := strrev(s);
4 writestr(result)
5 end

```

Текст out-файла:

```

1 .strings 1
2 0:  Hello
3 .code
4 0:  PUSH_STR  0
5 1:  STORE_STR 0
6 2:  LOAD_STR  0
7 3:  STRREV
8 4:  STORE_STR 1
9 5:  LOAD_STR  1
10 6:  PRINT_STR
11 7:  STOP

```

Вывод программы:

olleH

3.4 Некорректная программа

Текст исходного кода:

```

1 begin
2 num := 5;
3 text := "Test";
4 result := strcat(text, num);
5 writestr(result)
6 end

```

Вывод при запуске:

```

Line 4: string expression expected.
Line 4: identifier found while ')' expected.

```

3.5 Некорректный resize с -1

Текст исходного кода:

```
1 begin
2   s := "Hello";
3   cut := strresize(s, -1);
4   writestr(cut);
5   padded := strresize(s, 8);
6   writestr(padded)
7 end
```

Вывод при запуске:

Line 3: strresize: negative size is not allowed.

Заключение

В рамках данной лабораторной работы доработан компилятор языка SMilan согласно варианту: добавлены строки и операции со строками: конкатенация, поиск подстроки, замена подстроки, реверс, длина, `resize`; модифицирована контекстно-свободная грамматика языка, относящаяся ко 2 типу грамматик по иерархии Хомского. Грамматика языка Милан является LL(1)-грамматикой. В ходе выполнения работы выполнены поставленные задачи:

- Проанализирована работа операций со строками в различных языках программирования;
- Расширена форма Бэкуса-Наура контекстно-свободной грамматики языка MiLan;
- Построена синтаксическая диаграмма грамматики для расширенной версии языка MiLan;
- Компилятор SMilan и виртуальная машина успешно расширены поддержкой строк и операций над ними.

Достоинства программы:

- Разделение на два пространства имён переменных (`variables_`, `strVariables_`), что обеспечивает контроль типов;
- Обработка `\n`, `\t`, `\"`, `\\` прямо в `nextToken()` при чтении строкового литерала.

Недостатки программы:

- При переобъявлении переменной с тем же именем, но другим типом обе переменные будут существовать независимо, что приведёт к логически некорректной программе;
- `strfind` возвращает -1 при отсутствии подстроки, но целочисленный стек ВМ не разграничивает «нет результата» и «-1 как число».

Масштабированием программы может быть добавление других типов данных (`int`, `bool`), конструкций циклов (`for`).

Список литературы

- [1] Секция «Телематика». — Текст: электронный // tema.spbstu: [сайт]. — URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 27.03.2026)